SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I R.Nivetha / 1912260554 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nivetha

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: AI Drones for Emergency Medicine Delivery

• The drone has to reach a flooded area quickly, but it knows only a straight-line distance to the goal, gets blocked-path reports in pieces, and pays different costs depending on terrain and weather. So the real problem is finding a low-delay, low-risk route in a graph whose edges and costs keep changing. Speed matters, but a fast route that ends at a flooded dead end is worse than a slightly slower safe one.

i. Greedy Best-First Search

• Greedy search ranks nodes only by the heuristic, f(n) = h(n), the straight-line distance to the destination. The drone always moves to whichever reachable point looks closest to the goal and ignores what the trip has cost so far.

- **Strengths:** very fast, expands few nodes, needs little memory, and can re-decide quickly each time a new update arrives. That suits an urgent mission.
- **Weaknesses:** it is neither optimal nor reliably complete. Straight-line distance cannot see a flooded river or a blocked bridge in the way, so the drone can walk into a dead end and have to back out. It also ignores weather and terrain cost, so it may choose a short but dangerous stretch. Under uncertainty one wrong h-value is never corrected by the cost already paid.

ii. A* Search

• A* uses f(n) = g(n) + h(n). Here g(n) is the real accumulated cost from the base to n (distance weighted by terrain, wind and rain) and h(n) is the straight-line estimate of what remains. The g part stops the drone from running down cheap-looking but costly paths, and the h part keeps it pointed at the goal. If h never overestimates (for example, distance divided by the drone's best possible speed) and is consistent, A* returns the minimum-cost route. The price is more expanded nodes and more memory than Greedy.

iii. Impact of Heuristic Accuracy

Heuristic quality	Greedy Best-First	A*
Accurate (close to true cost)	Goes almost straight to the goal; works very well	Expands very few nodes and stays optimal

Weak / underestimates a lot	Wanders into detours and dead ends; route can be very poor	Still optimal, but expands many more nodes and is slower
Overestimates	Little change in behaviour; it was never guaranteed optimal anyway	Loses optimality and may miss the safest route
h = 0	Blind search	Becomes uniform-cost search

- In a flood, straight-line distance underestimates badly wherever the drone must go around water, so Greedy suffers more because it relies only on h, while A* stays correct and pays only in time.

iv. Effect of Dynamic Changes

- When a path is reported blocked, the plan built so far may be invalid and any stored g-values on it are out of date. Greedy recovers cheaply because it simply re-picks the best-looking neighbour from its current position, but it may walk back into the same trap. Plain A* has to recompute, which costs time when the update rate is high. Incremental versions such as D* Lite or LPA* repair only the affected part of the search instead of starting over. Partial information also means unknown edges must be treated cautiously, for example by adding a risk penalty to their cost.

v. Recommendation

- I recommend A* in a replanning form (D* Lite, or weighted A* with $f = g + w \cdot h$ and w of about 1.2 to 1.5). The reasons: (1) the cost g includes terrain and weather, so the route is the lowest real delay and risk and not just the shortest-looking one; (2) an admissible heuristic keeps the route optimal or within a known bound of optimal, which makes behaviour predictable and safe; (3) incremental repair and the weight w keep it fast enough for real time; (4) when computation time is nearly gone, the drone can fall back to Greedy for a quick decision. Greedy alone is too risky for medicine delivery in unreliable terrain.

Question 2: Smart City Traffic Signal Optimization

- Hundreds of intersections must be timed together to reduce waiting time, fuel use and congestion while traffic keeps changing. Nobody is looking for a path to a goal here. We want the best configuration, so this is an optimization problem and local search is the natural tool.

Formulation

- **State:** the full set of signal settings, one tuple (cycle length, green split, offset) per intersection.
- **Neighbour move:** change one setting at one intersection, such as adding 2 seconds of green or shifting an offset.
- **Objective:** minimize Cost = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion index), evaluated by a traffic simulator or live sensor data.
- **Constraints:** minimum green time, pedestrian crossing time, maximum cycle length.
- Traditional search (BFS, DFS, A*) is inefficient because there is no single goal state to test for, the path to a solution does not matter, and the space is astronomically large (if each of 300 intersections has only 10 possible settings, that is 10^300 combinations). It would also need huge memory, and traffic patterns change before the search finishes, making the answer outdated.

i. Hill Climbing

- Start from the current timing plan, evaluate neighbouring plans, move to the one that lowers the cost most, and repeat until no neighbour is better. It is simple, uses almost no memory and gives quick improvements, which is why it is attractive here. Its limitations are that it stops at the first peak it reaches, never makes a temporarily worse move, depends heavily on the starting plan, and does not adapt well when traffic shifts.

ii. Local Maxima, Plateaus and Ridges

- **Local optimum:** a green wave on the main road gives great flow on that corridor. Any single change makes things worse, so hill climbing stops, yet a different plan that also serves the cross streets could be much better overall.
- **Plateau:** during steady mid-day traffic, changing a green phase by a second or two does not change measured waiting time. With no signal about which direction is better, the search wanders or stops.
- **Ridge:** the best result needs neighbouring intersections to change together (for example, shifting offsets in a chain). Changing one intersection alone makes things worse, so single-step moves cannot climb along the ridge.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse plan with probability e^(-Δ/T), where T is a temperature that is lowered gradually. Early on, with high T, it explores widely and can climb out of local optima. Later, with low T, it behaves like hill climbing and settles into a good solution.

Genetic Algorithms keep a population of complete signal plans. Fitness is the inverse of cost. Good plans are selected, crossover combines parts of two plans (for instance, the corridor timing of one with the zone timing of another), and mutation adds random changes. Because many regions are explored in parallel and good sub-plans are recombined, ridges and local optima are far less of a trap.

iv. Recommendation

- I would use a Genetic Algorithm on a zone-by-zone basis, together with a short local refinement (hill climbing or low-temperature annealing) for live adjustment. The reasons: population evaluation can run in parallel, so it scales to hundreds of intersections; splitting the city into zones keeps each search space manageable; and the previous best plans can seed the next run, so when traffic patterns change the system adapts quickly instead of starting from scratch. Simulated Annealing alone is simpler and lighter, but it improves one solution at a time and scales less well. Hill climbing is kept only for the quick, small corrections between full optimizations.

Question 3: Autonomous Mars Rover

- The rover works in terrain it has never seen, senses only what is near it, and cannot be steered live from Earth because signals take several minutes each way. It has to decide on its own, keep learning the map as it moves, and avoid craters and rocks, which can end the mission if hit.

i. Why Online Search

- Offline search plans the whole route first and then executes it, which requires a complete and correct model. The rover has no map, so there is nothing to plan over. Also, actions are physical and often irreversible: it cannot try a path to see whether it works and then undo it. An online agent interleaves planning and acting: it plans with what it knows, takes a step, observes, and plans again.

ii. Challenges

- **Partial observability:** cameras and lidar have limited range, rocks hide what is behind them, dust and shadows add noise, and wheel slip makes

position estimates drift. The rover can never be sure what lies beyond what it sees, or whether a dark patch is a shadow or a crater.

- **Dynamic environment:** sand shifts, dust storms change visibility, lighting changes how the terrain looks, and battery and temperature limits change what the rover can afford to do.
- **Result:** plans go stale quickly, a path that looked safe may reveal a dead end, and a wrong decision can be permanent, so the rover must be cautious and keep replanning.

iii. Building and Updating the Internal Model

- The rover keeps a grid or occupancy map in which every cell is marked unknown, free or hazard, often with a probability. At each step it senses, updates the cells it can see (a Bayesian update, combined with localization/SLAM to correct its own position), and recalculates traversal costs. It also records what happened after each action (state, action, resulting state), which lets it avoid repeating mistakes, and updates its cost-to-goal estimates as it learns, as LRTA* does. The loop is: sense, update the map, plan on the current map, move a few steps, repeat.

iv. Comparison

Aspect	Uninformed (BFS/DFS)	Informed (A*, Greedy)	Online (online DFS, LRTA*, D* Lite)
Needs full map?	Yes (offline)	Yes (offline)	No, builds it as it goes
Guidance	None, blind	Heuristic toward goal	Heuristic plus live observations
Movement	Backtracking is physical travel, wasteful and risky	Jumps between frontier nodes, impractical for a real robot	Expands near its current position, so planning matches movement
Handles surprises	Poorly	Must replan from scratch	Repairs the plan incrementally
Safety	Low	Medium if the map is right	High, since it can add caution as it learns

v. Most Suitable Approach

- An online heuristic search with incremental replanning, such as D* Lite or LRTA*, backed by a safety layer. The heuristic (distance to the

sample site) keeps progress purposeful, replanning handles new observations cheaply, and the model grows as it explores, which covers uncertainty and adaptability. For safety, unknown cells are given extra cost, hazard cells get a margin around them, and the rover keeps enough energy to return to a safe position. Uninformed search wastes energy on physical backtracking and informed offline search depends on a map that does not exist.

Question 4: University Exam Timetable(CSP)

- This is a scheduling problem with many rules that must all hold at once. A timetable is acceptable only if every constraint is satisfied, which is exactly what a Constraint Satisfaction Problem describes: variables, the values they can take, and the rules between them.

i. Formulation

- **Variables:** each exam E1…En has two variables, its time slot (Slot_i) and its hall (Hall_i).
- **Domains:** Slot_i takes a value from the available time slots {1…T} and Hall_i from the halls {H1…Hm}. Domains are shortened at the start by faculty availability and special requirements.
- **Overlap constraint (binary):** if exams i and j share at least one student, then Slot_i ≠ Slot_j. This is the most common constraint and forms a conflict graph, like graph colouring.
- **Hall constraints:** two exams in the same slot cannot use the same hall (Hall_i ≠ Hall_j when Slot_i = Slot_j), and the number of students must not exceed the hall capacity.
- **Precedence constraint:** if subject A must come before B, then Slot_A < Slot_B.
- **Faculty constraint (unary):** slots where the examiner or invigilator is unavailable are removed from the domain, and one person cannot supervise two halls in the same slot.
- **Special accommodations:** students needing extra time, a separate room or an accessible hall restrict the hall domain to suitable rooms and may require longer slots or no back-to-back exams.

Hard constraints must hold. Softer wishes, such as not giving a student two exams on the same day, can be treated as preferences to minimize.

ii. Backtracking Search

• Backtracking assigns one variable at a time, for example Maths gets Slot 1 in Hall A. After each assignment it checks that no constraint is broken. If something is violated, it tries the next value, and if no value remains for a variable it backs up to the previous variable and changes that choice. For example, if Physics shares students with Maths and Chemistry and every slot is already used by those, Physics has no valid slot, so the search goes back and moves Maths or Chemistry. Plain backtracking is exponential, so it is combined with ordering heuristics: MRV (pick the variable with fewest remaining values), the degree heuristic (pick the one with most constraints) and least-constraining-value (try the value that rules out the fewest options).

iii. Constraint Propagation

• Forward checking removes values from the domains of unassigned neighbours as soon as a variable is assigned. If Maths is placed in Slot 1, Slot 1 is removed from every exam that shares students with Maths. If any domain becomes empty, the search fails immediately and backtracks instead of discovering the dead end many levels later. Precedence also prunes: if Programming is in Slot 2, Data Structures cannot use Slot 1 or 2. Stronger propagation such as arc consistency (AC-3) catches more conflicts, at a higher cost per step. Together with MRV, propagation can reduce the search by orders of magnitude.

iv. Real-World Challenges and Best Strategy

- Large size: thousands of students and hundreds of courses create a dense conflict graph.
- The problem may be over-constrained, so no perfect timetable exists and trade-offs are required.
- Late changes (a hall unavailable, a new accommodation request) force quick repairs.
- Fairness and student workload are soft goals that the basic CSP does not capture.

Best strategy: a hybrid. Use backtracking with MRV, degree and least-constraining-value ordering plus forward checking or AC-3 to build a valid timetable satisfying the hard constraints, splitting by department or exam block to keep each sub-problem small. Then use min-conflicts local search for improving soft constraints and for repairing the timetable after changes, since it fixes only the violated part and not the whole schedule. This scales

much better than pure backtracking while still guaranteeing that hard constraints are respected.

Question 5: Strategic Game AI (Maximum and Alpha-Beta)

- The AI plays a two-sided competitive game with a huge number of possible moves and has only a short time to answer. Looking at every possibility is impossible, so the challenge is to choose a good move within the time limit and not necessarily the perfect one.

i. Minimax Model

- The game is a tree. Each node is a game state and each edge is a move. The AI is MAX and the human is MIN (treated as a zero-sum contest, with the real-time game discretized into decision turns). Leaves, or nodes at the depth limit, get a utility or evaluation score. Values are passed upward: at MAX nodes take the highest child value, at MIN nodes the lowest. The AI then plays the move with the best value at the root. This assumes the opponent always plays the best reply, so the AI picks the move that is best even against a strong human.

ii. Computational Challenges

- Minimax explores O(b^d) nodes, where b is the branching factor and d the depth. With about 30 moves per state and a depth of 10, that is roughly 30^10, close to 6×10^{14} states, far beyond a time limit of a second or two. Other problems: each node requires evaluating a complex state, real-time play means the state is changing during thinking, hidden information such as fog of war makes states uncertain, and cutting the search at a fixed depth creates the horizon effect, where a danger just past the limit is missed.

iii. How Alpha-Beta Pruning Helps

- Alpha-beta carries two bounds through the search: α is the best value MAX is already guaranteed on the current path, and β is the best value MIN is guaranteed. If at any point $\alpha \geq \beta$, the rest of that branch cannot change the final decision and is skipped.

Example: MAX at the root, with MIN children. The first MIN child's leaves are 3 and 5, so its value is 3 and $\alpha = 3$ at the root. In the second MIN child, the first leaf is 2. MIN can already force a value of 2 or less, which is worse than the 3 MAX has, so MAX will never choose this branch and its other leaves are never examined. The result is identical to full minimax, only cheaper.

With good move ordering (best moves first) the cost drops from O(b^d) to about O(b^(d/2)), which lets the AI search roughly twice as deep in the same time. With bad ordering there is little saving, so ordering techniques such as killer moves, history heuristic and trying the previous iteration's best move first matter a lot.

iv. Evaluation Function

- Since the search cannot reach the end of the game, a function must score non-terminal states. A weighted linear form is cheap and effective:

Eval(s) = w1·Army + w2·Resources + w3·Map control + w4·Technology + w5·Base safety − w6·Enemy strength

- **Army strength:** number and quality of units weighted by their combat value and health. It is the most direct sign of who is winning a fight.
- **Resources and income:** minerals/gold gathered and the rate of gathering, because economy decides how much army can be built later.
- **Map control:** territory held and key positions occupied, which give vision and access to resources.
- **Technology and upgrades:** unlocked units and upgrades that give long-term advantage.
- **Base safety:** exposure of own buildings and worker units to attack.
- **Enemy strength:** the same factors for the opponent, subtracted.

- Weights can be set from expert knowledge and then tuned through self-play. A win scores very high and a loss very low. The function has to be fast because it is called at every leaf.

v. Strategy for Balancing Optimality and Real-Time Performance

- **Iterative deepening alpha-beta** with a strict time budget: search to depth 1, 2, 3 and so on, and when time runs out play the best move from the last completed depth. An answer is always available.

- **Move ordering and a transposition table** to maximize pruning and avoid re-evaluating repeated states.
- **Quiescence search** at the depth limit so the AI does not stop in the middle of a fight and misjudge it.
- **State abstraction and selective search:** group units, consider only promising moves (beam-style pruning) to reduce b.
- **Adaptive time use:** spend more time in critical moments and less when the position is calm. Where the branching factor is still too large, Monte Carlo Tree Search is a good alternative or addition. This gives optimal play where it matters and guarantees an answer inside the time limit.